Day 3 Agile and Scrum Foundations

Set 1: Online Grocery Delivery Application

Product Vision:

Product Vision – Online Grocery Delivery Application

For busy individuals and families who want to purchase groceries conveniently, the Online Grocery Delivery Application is a mobile application that allows customers to browse grocery products, search for items, add products to their cart, place orders, and track deliveries. Unlike traditional grocery shopping, this application provides convenient online shopping, easy checkout, real-time order tracking, and delivery notifications.

Scrum Roles & Anti-Patterns:

- **Product Owner:** Responsible for managing and prioritising the product backlog and ensuring the product delivers business value.
  Anti-pattern: A Product Owner who is unavailable or unable to make decisions.
- **Scrum Master:** Helps the team follow Scrum practices, facilitates Scrum events, and removes obstacles.
  Anti-pattern: A Scrum Master who behaves like a manager and assigns tasks to developers.
- **Developers:** Responsible for designing, developing, testing, and delivering a working product increment.
  Anti-pattern: Developers working in isolated groups without collaboration.

User Stories & Backlog:

Epic	**User Story Description**
Browse and Search	Browse grocery categories
Browse and Search	View product details
Browse and Search	Search for products
Browse and Search	Filter products by price
Cart and Checkout	Add products to cart
Cart and Checkout	Update product quantity

Cart and Checkout	Remove products from cart
Cart and Checkout	Place grocery order
Tracking	View order status
Tracking	Track delivery location
Tracking	View estimated delivery time
Tracking	Receive delivery notifications

INVEST Review & Story Rewriting:

INVEST stands for:

1. **Independent:** Each user story should be developed separately without depending heavily on other stories.
2. **Negotiable:** The details of the story can be discussed and modified by the team.
3. **Valuable:** Each story should provide value to the customer or business.
4. **Estimable:** The team should be able to estimate the effort required.
5. **Small:** Each story should be small enough to complete within one sprint.
6. **Testable:** Each story should have clear acceptance criteria to verify its completion.

Review of User Stories:

1. **Browse grocery categories:** Independent, valuable and testable.
2. **View product details:** Independent and testable.
3. **Search for products:** Valuable and testable.
4. **Filter products by price:** Small and testable.
5. **Add products to cart:** Independent and valuable.
6. **Update product quantity:** Small and testable.
7. **Remove products from cart:** Independent and testable.
8. **Place grocery order:** Valuable, but may be too large and can be split into smaller stories.
9. **View order status:** Independent and testable.
10. **Track delivery location:** Valuable and testable.
11. **View estimated delivery time:** Independent and testable.

12. **Receive delivery notifications:** Valuable and testable.

Story Rewriting – Place Grocery Order:

The original story, "Place grocery order", may involve multiple activities. To make it smaller and easier to estimate, it can be split into:

- **Story 1 – Enter Delivery Address:** As a customer, I want to enter my delivery address so that my groceries are delivered to the correct location.
- **Story 2 – Confirm and Place Order:** As a customer, I want to review and confirm my grocery order so that I can complete my purchase.

Result: The user stories are reviewed using INVEST principles. The oversized story is split into smaller, valuable and testable stories.

Acceptance Criteria (Given / When / Then):

Use these examples:

Story: Add products to cart

- Given a customer is viewing a grocery product, When the customer clicks Add to Cart, Then the product should be added to the cart.
- Given a product is out of stock, When the customer tries to add it to the cart, Then the application should display an out-of-stock message.

Story: Place grocery order

- Given the customer has products in the cart and a valid delivery address, When the customer confirms the order, Then the order should be placed successfully.
- Given the customer has not entered a valid delivery address, When the customer tries to place the order, Then the application should display a validation message.

Story: Track delivery location

- Given an order is out for delivery, When the customer opens the tracking page, Then the application should display the available delivery location.

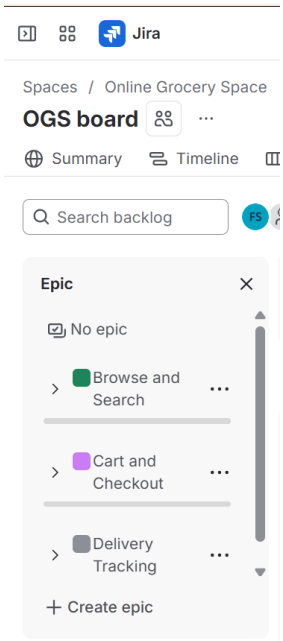
Estimation & Prioritization (MoSCoW & Story Points):

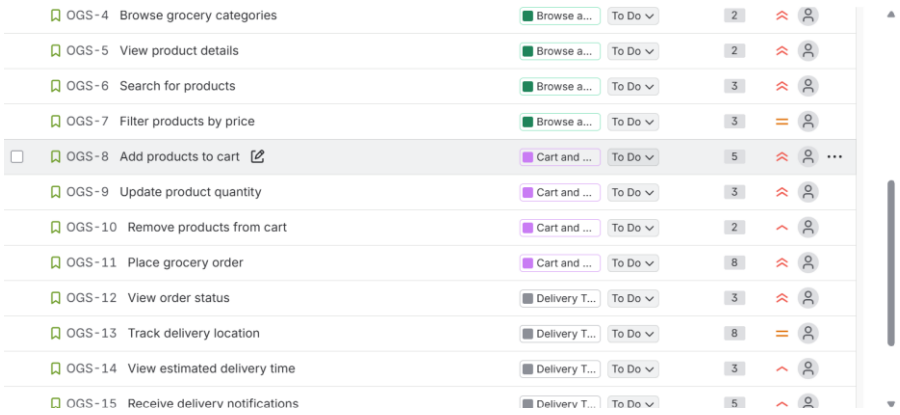
Reference Story Chosen: FCG-105 (Cart Update) = 2 Story Points.

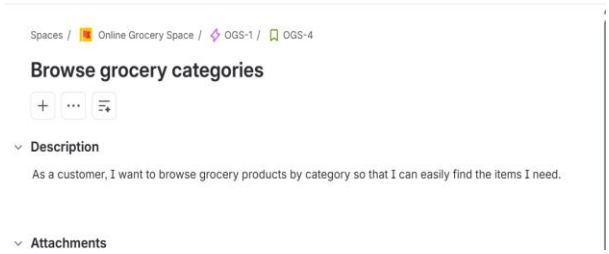
Story	**Priority**	**Points**
Browse grocery categories	High	3
View product details	High	2
Search for products	High	5

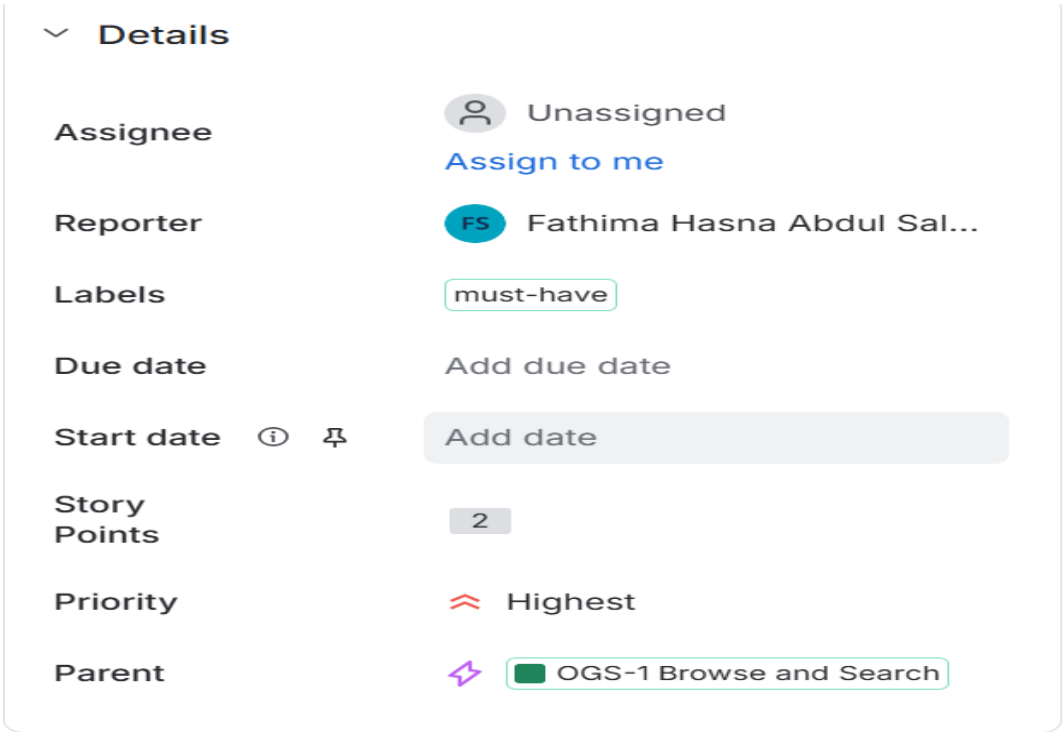
Filter Product by price	Medium	2
Add products to cart	High	3
Update product quantity	High	2
Remove products from cart	Medium	2
Place grocery order	High	5
View order status	High	3
Track delivery location	Medium	5
View estimated delivery time	Medium	2
Receive delivery notifications	Medium	3

JIRA Board & Quality Criteria:









- **Definition of Ready (DoR):** Story in 'As a / I want / So that' format, clear acceptance criteria, and estimated points.
- **Definition of Done (DoD):** Code completed, unit tests passed (80%+ coverage), QA tested, and approved by Product Owner.

Set 2: Insurance Claims Portal

Product Vision:

For policyholders who need to submit and track insurance claims, the Insurance Claims Portal is a web application that allows users to register claims, upload supporting documents, check claim status, and receive settlement notifications. It simplifies the claim process, reduces paperwork, improves transparency, and helps customers track their claims conveniently.

Waterfall vs. Agile Comparison:

Waterfall relies on rigid, sequential phases where changes late in the project cause major delays. Insurance regulations and user needs change frequently. Agile allows releasing a core working MVP (Claim Submission) in 4 weeks and iteratively adding feature improvements based on real feedback.

User Stories & Backlog:

Epic	User Story Description
Claim Registration	Register a New Insurance Claim
Claim Registration	Enter Policy Details
Claim Registration	Receive Claim Reference Number
Document Upload	Upload Supporting Documents
Document Upload	View Uploaded Documents
Document Upload	Replace an Uploaded Document
Claim Tracking	View Claim Status
Claim Tracking	View Claim Processing Updates
Claim Tracking	View Pending Information Required
Settlement Notifications	Receive Claim Status Notifications
Settlement Notifications	Receive Settlement Approval Notification
Settlement Notifications	View Settlement Details

INVEST Review & Vertical Splitting:

1. Independent: Each story should be developed separately.
2. Negotiable:Requirements can be discussed and modified.
3. Valuable:Each story should provide business or customer value.
4. Estimable:The team should be able to estimate the work.
5. Small:The story should be small enough to complete in a sprint.

6. Testable: The story should have clear acceptance criteria.

Review of User Stories:

1. Register a New Insurance Claim – Valuable and testable.
2. Enter Policy Details – Independent and testable.
3. Receive Claim Reference Number – Small and valuable.
4. Upload Supporting Documents – Valuable, but may need to be split if too large.
5. View Uploaded Documents – Independent and testable.
6. Replace an Uploaded Document – Small and testable.
7. View Claim Status – Independent and valuable.
8. View Claim Processing Updates – Valuable and testable.
9. View Pending Information Required – Independent and testable.
10. Receive Claim Status Notifications – Valuable and testable.
11. Receive Settlement Approval Notification – Small and testable.
12. View Settlement Details – Independent and valuable.

Story Rewriting – Upload Supporting Documents:

If uploading documents is too large, it can be split into:

Story 1 – Upload Claim Documents: As a policyholder, I want to upload supporting documents so that I can provide evidence for my insurance claim.

Story 2 – Validate Uploaded Documents: As a policyholder, I want the system to validate uploaded documents so that incorrect file formats and oversized files are rejected.

Result: The stories are reviewed using INVEST principles. Oversized stories can be divided into smaller, independent and testable stories.

Acceptance Criteria:

Story 1 – Register a New Insurance Claim

- Given a policyholder is logged in, When the policyholder submits valid claim details, Then the system should register the claim successfully.
- Given required claim details are missing, When the policyholder submits the form, Then the system should display validation errors.

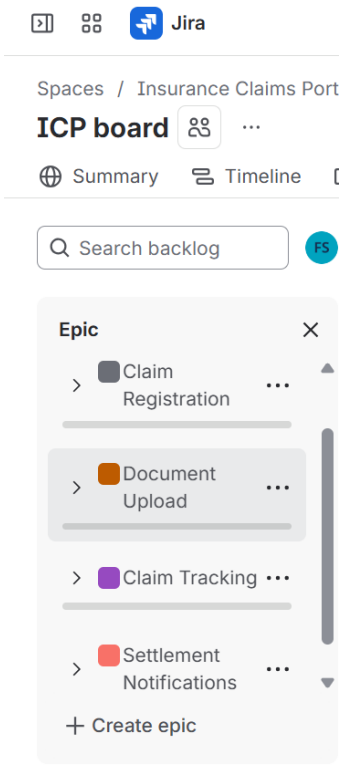
Story 2 – Enter Policy Details

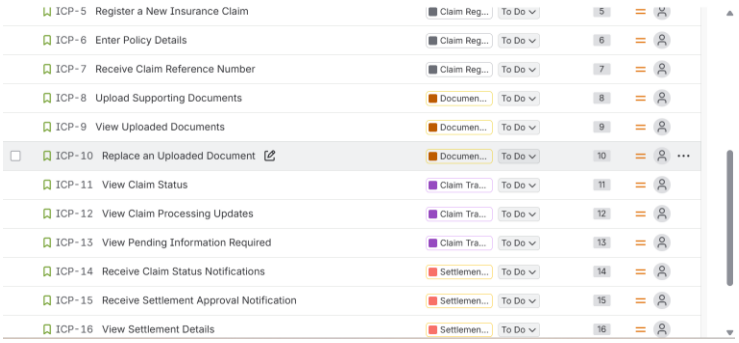
- Given the claim registration page is open, When the policyholder enters a valid policy number, Then the system should verify and display the policy details.
- Given the policyholder enters an invalid policy number, When the policyholder submits it, Then the system should display an error message and prevent claim submission.

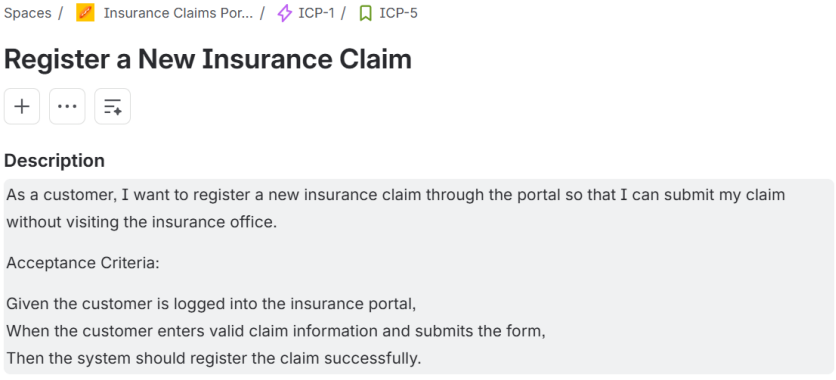
Story 3 – Receive Claim Reference Number

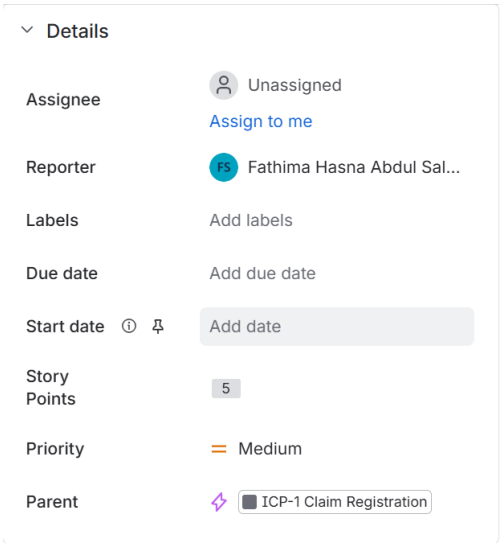
- Given a claim has been submitted successfully, When the system registers the claim, Then it should generate and display a unique claim reference number.

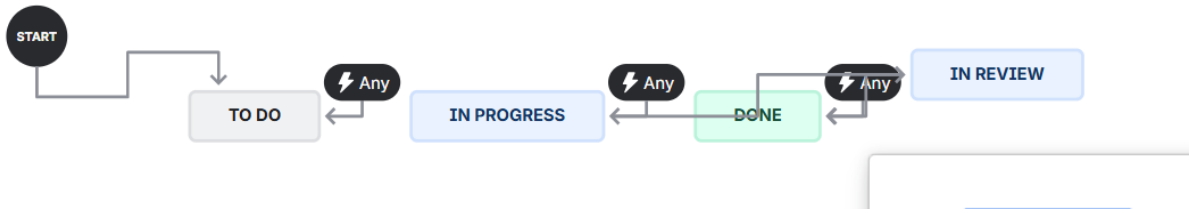
JIRA Board Screenshot & Scrum Overview:











- Roles: Product Owner, Scrum Master, Developers.
- Events: Sprint Planning, Daily Scrum, Sprint Review, Retrospective.
- Artifacts: Product Backlog, Sprint Backlog, Increment.

Set 3: Hospital Appointment Booking Application

Product Vision:

For patients who want to book doctor appointments easily, the Hospital Appointment Booking Application is a mobile application that allows patients to search for doctors, view available appointment slots, book appointments, reschedule or cancel bookings, and receive appointment reminders. It simplifies the appointment process, reduces waiting time, and helps hospitals manage doctor schedules efficiently.

Stakeholder Mapping Matrix:

Stakeholder	**Scrum Role**	**Responsibilities**
Hospital Chief Medical Officer	Product Owner	Defines feature requirements and clinical priorities.
Agile Coach / Project Lead	Scrum Master	Facilitates Scrum framework and removes blockers.
Mobile Developers & Testers	Developer	Builds, tests, and deploys mobile app features.
Patients & Visitors	Customer	End users booking doctor appointments.
Hospital Desk Staff	Customer / End User	Manages daily doctor schedules on-site.

User Stories & Backlog:

Epic	User Story Description
Appointment Booking	Search for Doctors
Appointment Booking	View Doctor Availability
Appointment Booking	Book an Appointment
Rescheduling and Cancellation	View My Appointments
Rescheduling and Cancellation	Reschedule an Appointment
Rescheduling and Cancellation	Cancel an Appointment
Reminders	Receive Appointment Confirmation
Reminders	Receive Appointment Reminder
Reminders	View Appointment Notification Details
Doctor Schedule	Set Doctor Availability
Doctor Schedule	Manage Appointment Slots
Doctor Schedule	Block Unavailable Time Slots

INVEST Review Refinements:

INVEST stands for:

1. **Independent:** Each story should be developed separately.
2. **Negotiable:** Requirements can be discussed and modified.

3. **Valuable:** Each story should provide value to patients or the hospital.
4. **Estimable:** The team should be able to estimate the effort.
5. **Small:** Each story should be small enough to complete within a sprint.
6. **Testable:** Each story should have clear acceptance criteria.

Review of User Stories:

1. Search for Doctors – Independent, valuable and testable.
2. View Doctor Availability – Valuable and testable.
3. Book an Appointment – Valuable, but may need to be split if too large.
4. View My Appointments – Independent and testable.
5. Reschedule an Appointment – Valuable and testable.
6. Cancel an Appointment – Independent and testable.
7. Receive Appointment Confirmation – Small and testable.
8. Receive Appointment Reminder – Valuable and testable.
9. View Appointment Notification Details – Independent and testable.
10. Set Doctor Availability – Valuable and testable.
11. Manage Appointment Slots – Independent and estimable.
12. Block Unavailable Time Slots – Small and testable.

Story Rewriting – Book an Appointment:

If booking an appointment is too large, it can be split into:

- **Story 1 – Select Appointment Slot:** As a patient, I want to select an available appointment slot so that I can choose a convenient date and time.
- **Story 2 – Confirm Appointment Booking:** As a patient, I want to confirm my selected appointment so that my booking is completed successfully.

Result: The user stories are reviewed using INVEST principles. Oversized stories can be divided into smaller, independent and testable stories.

Acceptance Criteria (Booking & Slot Conflict):

Story 3 – Book an Appointment

- Given a patient has selected an available appointment slot, When the patient confirms the booking, Then the appointment should be booked successfully.
- Given a selected appointment slot has already been taken, When the patient tries to book it, Then the system should display an error message saying "Slot no longer available" and ask the patient to select another slot.

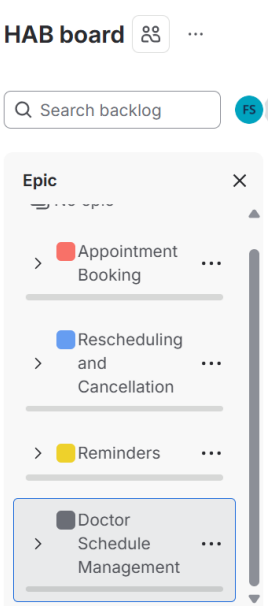
Story 6 – Cancel an Appointment

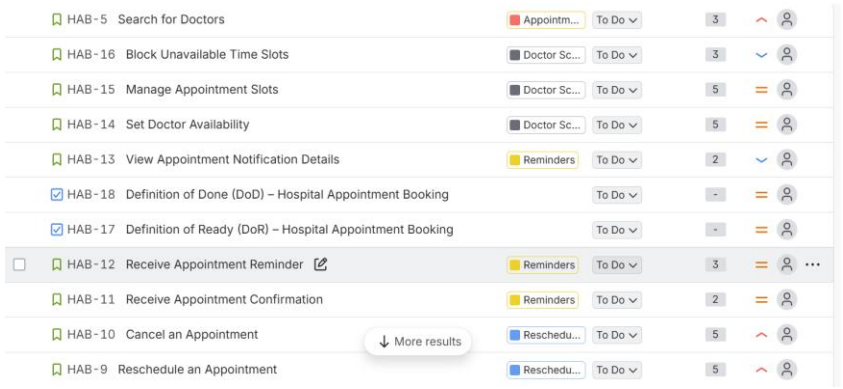
- Given a patient has an upcoming appointment, When the patient confirms cancellation, Then the appointment should be cancelled successfully and the slot should become available.
- Given an appointment is already cancelled, When the patient tries to cancel it again, Then the system should display an appropriate message.

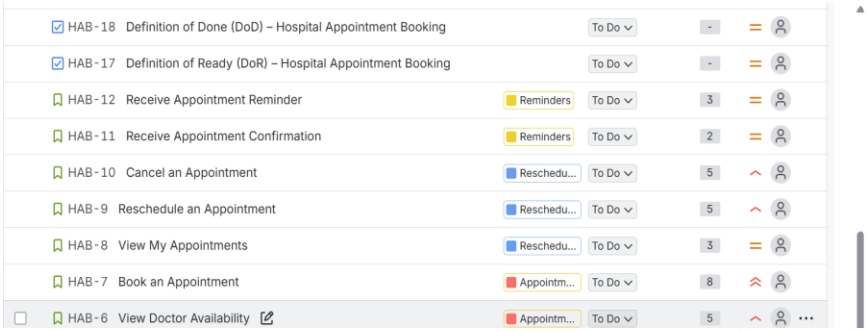
Prioritization & Story Points vs. Hours:

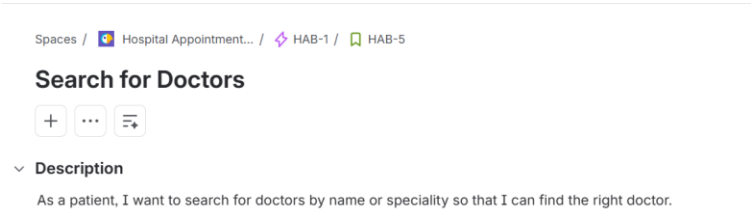
- **Story Points:** Story Points measure relative effort, complexity, and uncertainty. They do not change based on who completes the work.
- **Ideal Hours:** Hours measure absolute elapsed time, which varies depending on developer experience level.

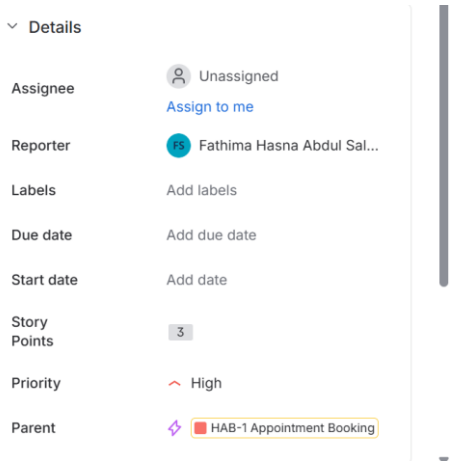
JIRA Board & DoR/DoD Checklists:

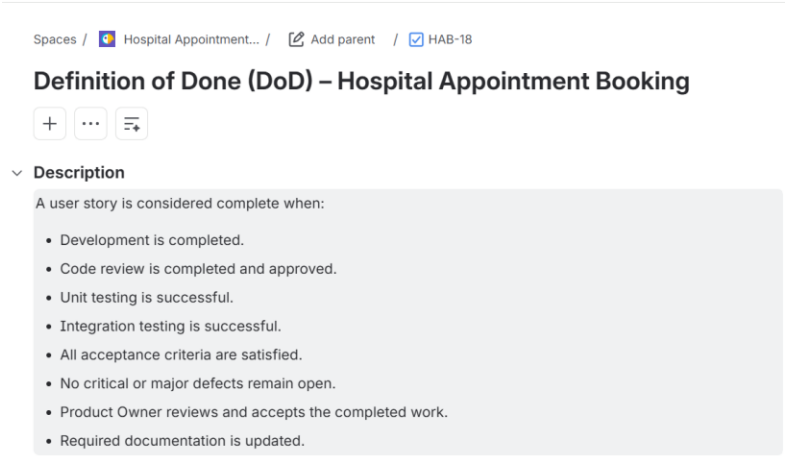


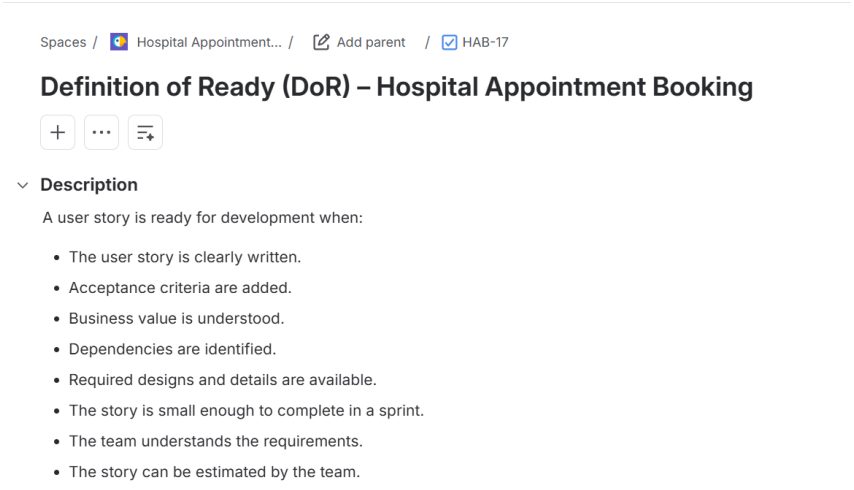







Details
Assignee
Unassigned
Assign to me
Reporter
Fathima Hasna Abdul Sal...
Labels
Add labels
Due date
Add due date
Start date
Add date
Story Points
3
Priority
High
Parent
HAB-1 Appointment Booking


Spaces / Hospital Appointment... / Add parent / HAB-18
Definition of Done (DoD) – Hospital Appointment Booking
Description
A user story is considered complete when:
Development is completed.
Code review is completed and approved.
Unit testing is successful.
Integration testing is successful.
All acceptance criteria are satisfied.
No critical or major defects remain open.
Product Owner reviews and accepts the completed work.
Required documentation is updated.


Spaces / Hospital Appointment... / Add parent / HAB-17
Definition of Ready (DoR) – Hospital Appointment Booking
Description
A user story is ready for development when:
The user story is clearly written.
Acceptance criteria are added.
Business value is understood.
Dependencies are identified.
Required designs and details are available.
The story is small enough to complete in a sprint.
The team understands the requirements.
The story can be estimated by the team.